

Ecosistema

Librerie esterne

Introduzione all'ecosistema di librerie Python.

Perché ti servono le librerie esterne

Python da solo è già potente, ma la sua vera forza è la comunità: milioni di programmatori di tutto il mondo hanno creato librerie che risolvono problemi comuni. Invece di scrivere tutto da zero, puoi usare il loro lavoro.

Vuoi creare un sito web? C'è una libreria. Fare intelligenza artificiale? C'è una libreria. Analizzare dati come in Excel? C'è una libreria. Sviluppare un videogioco? C'è una libreria.

Come usare una libreria esterna

Il processo è sempre lo stesso in tre passi:

Passo 1: Installa la libreria (una volta sola, dal terminale)

```
pip install requests
```

Passo 2: Importa la libreria nel tuo codice

Passo 3: Usa le funzionalità che offre

```
risposta = requests.get("https://api.github.com")
print(risposta.status_code)    # 200 = tutto ok
```

Le categorie principali

Web e internet

- **requests** — scarica dati da internet, interagisce con API web. La libreria Python più scaricata in assoluto.
 - **flask** — crea siti web e API con pochissimo codice. Ottimo per imparare.
 - **django** — framework web completo, usato da Instagram e Pinterest.
 - **fastapi** — API moderne e velocissime, con documentazione automatica.
-

Dati e scienza

- **numpy** — calcoli matematici su grandi quantità di numeri, molto più veloce delle liste normali.
 - **pandas** — analisi di dati tabellari (come Excel, ma programmabile e automatizzabile).
 - **matplotlib** — crea grafici: istogrammi, torte, linee, dispersione, ecc.
 - **scikit-learn** — algoritmi di machine learning pronti da usare.
 - **tensorflow** / **pytorch** — deep learning e reti neurali.
-

Automazione

- **selenium** — controlla un browser in modo automatico (utile per testare siti web o automatizzare task ripetitivi).
 - **beautifulsoup4** — legge e analizza il codice HTML di pagine web per estrarne i dati.
 - **pyautogui** — muove il mouse e preme tasti in modo automatico.
 - **schedule** — esegue del codice automaticamente a orari programmati.
-

Grafica e giochi

- **pygame** — sviluppa videogiochi 2D in Python.
- **pillow** — apri, modifica e salva immagini (ridimensiona, ritaglia, applica filtri, ecc.).
- **tkinter** — crea finestre con pulsanti, caselle di testo e menu (già incluso in Python).

- **PyQt** / **PySide** — interfacce grafiche più avanzate e professionali.
-

Utilità

- **tqdm** — aggiunge una barra di avanzamento ai tuoi cicli, così vedi quanto manca.
 - **rich** — stampa testo colorato e tabelle belle nel terminale.
 - **loguru** — sistema di log semplice e con output colorato.
 - **python-dotenv** — legge variabili di configurazione da un file `.env`.
-

Esempio: scaricare dati dal web con **requests**

```
pip install requests
```

```
# Chiede dati a un'API pubblica di prova
risposta = requests.get("https://jsonplaceholder.typicode.com/posts/1")

if risposta.status_code == 200:    # 200 = successo
    dati = risposta.json()        # Converte la risposta JSON in un
    dizionario Python
    print(f"Titolo: {dati['title']}")
    print(f"Testo: {dati['body'][:50]}...") # Solo i primi 50 caratteri
else:
    print(f"Errore: {risposta.status_code}")
```

Esempio: barra di avanzamento con **tqdm**

```
pip install tqdm
```

```
from tqdm import tqdm

# Aggiunge una barra di avanzamento a qualsiasi ciclo for
for i in tqdm(range(100), desc="Elaborazione"):
    time.sleep(0.05)  # Simula un'operazione che richiede tempo
```

Output nel terminale:

```
Elaborazione: 100%|██████████| 100/100 [00:05<00:00, 20.00it/s]
```

Come trovare la libreria giusta

1. Controlla prima se esiste qualcosa nella **libreria standard** di Python (senza installare niente)
2. Cerca su Google: "python libreria per [quello che ti serve]"
3. Guarda le stelle su GitHub — più stelle = più popolare e affidabile
4. Controlla la data dell'ultimo aggiornamento — una libreria aggiornata di recente è un buon segno
5. Leggi brevemente la documentazione ufficiale

Il consiglio fondamentale: **non reinventare la ruota**. Se qualcosa è già stato fatto da qualcun altro in modo professionale, usalo.

NumPy

Introduzione alla libreria NumPy per il calcolo numerico in Python.

Perché è utile usare NumPy

Le liste Python vanno bene per la maggior parte dei casi. Ma quando hai bisogno di fare **calcoli matematici su migliaia o milioni di numeri**, le liste diventano lente. NumPy risolve questo problema: fa le stesse operazioni ma in modo molto più veloce.

NumPy è alla base di quasi tutto l'ecosistema scientifico di Python: pandas, matplotlib, scikit-learn usano tutti NumPy internamente.

Installazione e importazione

```
pip install numpy
```

Per convenzione, NumPy si importa sempre con il soprannome `np` :

```
import numpy as np
```

Gli array di NumPy

La struttura centrale di NumPy è l'**array** (`ndarray`) — simile a una lista Python, ma pensata per i calcoli numerici.

```
# Crea un array da una lista Python
a = np.array([1, 2, 3, 4, 5])
print(a)           # [1 2 3 4 5]
print(type(a))     # <class 'numpy.ndarray'>
print(a.dtype)     # int64 – il tipo dei dati (tutti interi a 64 bit)
print(a.shape)     # (5,) – la forma: 5 elementi in una dimensione

# Crea una matrice (array a due dimensioni)
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(m)
print(m.shape)     # (3, 3) – 3 righe, 3 colonne
```

Creare array già pronti

```
print(np.zeros(5))      # [0. 0. 0. 0. 0.] – cinque zeri
print(np.ones(4))       # [1. 1. 1. 1.] – quattro uni
print(np.zeros((2, 3))) # matrice 2x3 di zeri
print(np.eye(3))        # matrice identità 3x3 (uno sulla diagonale)
print(np.arange(0, 10, 2)) # [0 2 4 6 8] – come range(), ma per array
print(np.linspace(0, 1, 5)) # [0. 0.25 0.5 0.75 1.] – 5 punti equidistanti tra 0 e 1
print(np.random.rand(3)) # 3 numeri decimali casuali tra 0 e 1
```

La potenza delle operazioni vettoriali

Con le liste Python, per moltiplicare ogni elemento per 2 dovresti scrivere un ciclo. Con NumPy, le operazioni si applicano **all'intero array in una volta sola**:

```

a = np.array([1, 2, 3, 4, 5])

# Operazioni su tutti gli elementi – senza scrivere cicli!
print(a + 10)    # [11 12 13 14 15]
print(a * 2)     # [ 2  4  6  8 10]
print(a ** 2)    # [ 1  4  9 16 25]
print(a > 3)     # [False False False  True  True]

# Operazioni tra due array (elemento per elemento)
b = np.array([10, 20, 30, 40, 50])
print(a + b)     # [11 22 33 44 55]
print(a * b)     # [ 10  40  90 160 250]

```

Questo approccio è detto **vettorializzazione**: invece di fare le operazioni una alla volta con un ciclo, NumPy le fa tutte insieme in modo ottimizzato.

Funzioni matematiche integrate

```

a = np.array([1, 4, 9, 16, 25])

print(np.sqrt(a))    # [1. 2. 3. 4. 5.] – radice quadrata di ogni elemento
print(np.sum(a))     # 55    – somma di tutti gli elementi
print(np.mean(a))    # 11.0  – media
print(np.min(a))     # 1     – valore minimo
print(np.max(a))     # 25    – valore massimo
print(np.std(a))     # deviazione standard

```

Accedere agli elementi

```
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(m[0])          # [1 2 3]          ← prima riga intera
print(m[1, 2])       # 6                ← riga 1, colonna 2
print(m[:, 0])       # [1 4 7]         ← tutta la colonna 0
print(m[0:2, 1:3])   # [[2 3] [5 6]]   ← un sotto-blocco (righe 0-1, colonne 1-2)
```

NumPy vs liste Python: la differenza di velocità

```
n = 1_000_000    # Un milione di numeri

# Metodo 1: lista Python con ciclo
lista = list(range(n))
inizio = time.time()
risultato = [x * 2 for x in lista]
print(f"Lista Python: {time.time() - inizio:.3f} secondi")

# Metodo 2: array NumPy
array = np.arange(n)
inizio = time.time()
risultato = array * 2
print(f"NumPy: {time.time() - inizio:.3f} secondi")
```

NumPy è tipicamente **10-100 volte più veloce** delle liste Python per operazioni numeriche. Su un milione di elementi, la differenza è notevole.

Quando usare NumPy vs liste

Situazione	Usa...
Dati misti (stringhe, numeri, oggetti)	Lista Python
Calcoli matematici su molti numeri	NumPy
Semplice raccolta di elementi	Lista Python
Matrici, algebra lineare, statistiche	NumPy
Qualsiasi libreria scientifica (pandas, matplotlib)	NumPy (lo usano internamente)

Pandas

Introduzione alla libreria Pandas per l'analisi dei dati in Python.

Perché è utile usare Pandas

Immagina di avere un foglio Excel con migliaia di righe e di dover trovare tutti gli studenti con media superiore all'8, ordinarli per cognome, e salvare il risultato in un nuovo file. A mano richiederebbe ore.

Con **Pandas** lo fai in poche righe di codice Python — e puoi ripeterlo automaticamente ogni volta che i dati cambiano.

Cos'è Pandas?

Pandas è la libreria Python più usata per lavorare con **dati tabellari** — cioè dati organizzati in righe e colonne, come un foglio Excel o una tabella di database.

Le due strutture principali sono:

- **Series**: una singola colonna di dati (come una lista con etichette)
- **DataFrame**: una tabella completa con righe e colonne

Installazione e importazione

```
pip install pandas
```

Per convenzione, Pandas si importa con il soprannome `pd` :

Series: una colonna di dati

```
# Crea una Series da una lista
voti = pd.Series([8, 7, 9, 6, 10])
print(voti)
# 0      8
# 1      7
# 2      9
# 3      6
# 4     10
# dtype: int64

# Crea una Series con etichette personalizzate (invece dei numeri 0, 1, 2...)
voti_classe = pd.Series(
    [8, 7, 9, 6],
    index=["Alice", "Bob", "Carlo", "Diana"]
)
print(voti_classe["Alice"])    # 8
print(voti_classe.mean())     # 7.5 - media
```

DataFrame: la tabella di dati

Un **DataFrame** è come un foglio Excel: ha righe e colonne con nomi.

```
# Crea un DataFrame da un dizionario (ogni chiave è una colonna)
studenti = pd.DataFrame({
    "nome": ["Alice", "Bob", "Carlo", "Diana"],
    "eta":  [16, 15, 17, 16],
    "voto": [8, 7, 9, 6],
    "citta": ["Roma", "Milano", "Roma", "Napoli"]
})

print(studenti)
```

Output:

	nome	eta	voto	citta
0	Alice	16	8	Roma
1	Bob	15	7	Milano
2	Carlo	17	9	Roma
3	Diana	16	6	Napoli

Accedere ai dati

```
# Selezionare una colonna (come accedere a un dizionario)
print(studenti["nome"])

# Selezionare più colonne
print(studenti[["nome", "voto"]])

# Selezionare righe per numero (iloc = integer location)
print(studenti.iloc[0])      # Prima riga (indice 0)
print(studenti.iloc[0:2])    # Righe 0 e 1

# Selezionare un singolo valore (riga, colonna)
print(studenti.iloc[0, 2])    # Riga 0, colonna 2 → 8 (il voto di Alice)
```

Filtrare i dati

Qui Pandas diventa potente: puoi selezionare solo le righe che soddisfano una condizione.

```
# Tutti gli studenti con voto maggiore o uguale a 8
promossi = studenti[studenti["voto"] >= 8]
print(promossi)

# Tutti gli studenti di Roma
romani = studenti[studenti["citta"] == "Roma"]
print(romani)

# Filtri combinati: voto >= 7 E età >= 16
risultato = studenti[(studenti["voto"] >= 7) & (studenti["eta"] >= 16)]
print(risultato)
```

Statistiche in un colpo solo

```
print(studenti["voto"].mean())    # 7.5  - media dei voti
print(studenti["voto"].max())     # 9    - voto più alto
print(studenti["voto"].min())     # 6    - voto più basso
print(studenti["voto"].std())     # deviazione standard

# Riepilogo statistico di tutte le colonne numeriche
print(studenti.describe())
```

Leggere e scrivere file

Una delle funzionalità più utili: Pandas legge e scrive file CSV ed Excel direttamente.

```
# Leggere un file CSV (valori separati da virgole)
df = pd.read_csv("dati.csv")

# Scrivere su CSV (index=False evita di scrivere i numeri di riga)
studenti.to_csv("studenti_export.csv", index=False)

# Leggere un file Excel
df_excel = pd.read_excel("dati.xlsx")

# Scrivere su Excel
studenti.to_excel("studenti_export.xlsx", index=False)
```

Operazioni comuni

```
# Aggiungere una nuova colonna calcolata
studenti["promosso"] = studenti["voto"] >= 6 # True o False per ogni riga
print(studenti)

# Ordinare per voto (dal più alto al più basso)
ordinati = studenti.sort_values("voto", ascending=False)
print(ordinati)

# Raggruppare e calcolare statistiche per gruppo
# (media dei voti per ogni città)
per_citta = studenti.groupby("citta")["voto"].mean()
print(per_citta)

# Informazioni generali sul DataFrame
print(studenti.shape) # (4, 4) - 4 righe, 4 colonne
print(studenti.dtypes) # tipo di dati in ogni colonna
```

Uso di pacchetti (pip)

Come installare e gestire pacchetti Python con pip.

Qual è la funzione di pip

Python ha già molte funzionalità integrate, ma la sua vera forza è l'enorme numero di **librerie esterne** create dalla comunità: strumenti per fare grafica, analisi dati, intelligenza artificiale, giochi, siti web e molto altro.

pip è lo strumento che ti permette di scaricare e installare queste librerie in pochi secondi.

Cos'è pip?

pip è il "gestore di pacchetti" ufficiale di Python. È come un negozio online: ti permette di cercare e installare gratuitamente oltre 400.000 librerie da **PyPI** (il deposito ufficiale di Python, all'indirizzo pypi.org).

Pip è già incluso con Python — non devi installarlo separatamente.

Verificare che pip funzioni

Apri il terminale (o il prompt dei comandi su Windows) e digita:

```
pip --version
# pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.x)
```

Se hai sia Python 2 che Python 3, potrebbe servire **pip3** :

```
pip3 --version
```

Installare una libreria

```
pip install nome-libreria
```

Esempio: installare `requests` , la libreria più usata per fare richieste al web:

```
pip install requests
```

Installare una versione specifica (utile quando un progetto richiede esattamente una certa versione):

```
pip install requests==2.31.0
```

Aggiornare una libreria all'ultima versione disponibile:

```
pip install --upgrade requests
```

Disinstallare una libreria

```
pip uninstall requests
```

Vedere le librerie installate

```
pip list
```

Output di esempio:

Package	Version
-----	-----
pip	24.0
requests	2.31.0
numpy	1.26.0

Il file `requirements.txt` : condividere le dipendenze

Quando condividi un progetto con altri, devono sapere quali librerie installare. Il file `requirements.txt` elenca tutto quello che serve.

Creare il file (salva le librerie attualmente installate):

```
pip freeze > requirements.txt
```

Contenuto tipico di `requirements.txt` :

```
requests==2.31.0  
numpy==1.26.0  
pandas==2.1.0
```

Installare tutto da un `requirements.txt` (utile quando ricevi un progetto da qualcun altro):

```
pip install -r requirements.txt
```

Ottenere informazioni su una libreria

```
pip show requests
```

Output:

```
Name: requests
Version: 2.31.0
Summary: Python HTTP for Humans.
Home-page: https://requests.readthedocs.io
```

Librerie popolari da conoscere

Libreria	A cosa serve
<code>requests</code>	Scaricare dati dal web, fare richieste HTTP
<code>numpy</code>	Calcoli matematici veloci su grandi quantità di numeri
<code>pandas</code>	Analisi e manipolazione di dati (come Excel ma programmabile)
<code>matplotlib</code>	Creare grafici e visualizzazioni
<code>flask</code>	Creare siti web semplici
<code>django</code>	Creare siti web completi
<code>pillow</code>	Modificare immagini
<code>pygame</code>	Sviluppare videogiochi 2D
<code>beautifulsoup4</code>	Estrarre dati da pagine web

Buone pratiche

- Usa sempre un **ambiente virtuale** per ogni progetto (vedi il capitolo successivo)

- Tieni aggiornato il file `requirements.txt` quando aggiungi librerie
 - Prima di installare una libreria sconosciuta, verifica che sia affidabile: guarda quante stelle ha su GitHub e quanti download ha su PyPI
-

Ambiente virtuale

Come isolare le dipendenze dei progetti Python con gli ambienti virtuali.

Perché ti serve un ambiente virtuale

Immagina di avere due progetti Python sul tuo computer:

- **Progetto A** funziona con la versione 2.25 della libreria `requests`
- **Progetto B** ha bisogno della versione 2.31 della stessa libreria

Se installi tutto "globalmente" (cioè direttamente sul tuo Python di sistema), i due progetti entrano in conflitto: puoi avere una versione sola, e l'altra non funziona.

Gli **ambienti virtuali** risolvono questo problema: ogni progetto ha la sua "bolla" separata con le proprie librerie, nella versione giusta.

Cos'è un ambiente virtuale?

Un ambiente virtuale è una **cartella isolata** che contiene una copia di Python e di tutte le librerie installate per quel progetto. È separata dal Python del sistema e dagli altri progetti.

Pensa a uno zaino: ogni studente ha il suo, con i propri libri e materiali. Anche se due studenti usano lo stesso manuale, le loro copie sono indipendenti.

Creare un ambiente virtuale

Python include già il modulo `venv` per creare ambienti virtuali. Nella cartella del tuo progetto, esegui:

```
python -m venv venv
```

Questo crea una sottocartella chiamata `venv` con tutto il necessario. Puoi darle qualsiasi nome, ma `venv` o `.venv` sono le convenzioni più usate.

Attivare l'ambiente virtuale

Prima di poterlo usare, devi "attivarlo":

Su macOS e Linux:

```
source venv/bin/activate
```

Su Windows:

```
venv\Scripts\activate
```

Quando l'ambiente è attivo, il nome appare all'inizio del prompt del terminale:

```
(venv) $
```

Da questo momento, qualsiasi `pip install` installerà le librerie **solo in questo ambiente**, non sul sistema globale.

Disattivare l'ambiente virtuale

Quando hai finito di lavorare al progetto:

```
deactivate
```

Il prompt torna alla normalità, senza `(venv)`.

Flusso di lavoro tipico

Ecco la sequenza completa per un nuovo progetto:

```
# 1. Crea la cartella del progetto
mkdir mio-progetto
cd mio-progetto

# 2. Crea l'ambiente virtuale
python -m venv venv

# 3. Attivalo
source venv/bin/activate      # macOS/Linux
# venv\Scripts\activate      # Windows (alternativa)

# 4. Installa le librerie necessarie
pip install requests pandas

# 5. Lavora sul progetto...

# 6. Salva le dipendenze in un file per condividerle
pip freeze > requirements.txt
```

Ripristinare un progetto su un altro computer

Quando qualcuno riceve il tuo progetto (o tu stesso lo riapri su un altro computer):

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt  # Installa tutte le librerie elencate nel
file
```

Non aggiungere `venv` a Git

La cartella `venv` è pesante (può occupare centinaia di megabyte) e si può sempre ricreare con `pip install -r requirements.txt`. Non ha senso caricarla su Git.

Aggiungi queste righe al file `.gitignore` del tuo progetto:

```
venv/  
.venv/  
__pycache__/  
*.pyc
```

Strumenti alternativi (per il futuro)

Strumento	Descrizione
<code>venv</code>	Già incluso in Python — ideale per iniziare
<code>virtualenv</code>	Come <code>venv</code> ma con qualche funzione in più
<code>conda</code>	Molto usato in ambito scientifico (Anaconda)
<code>poetry</code>	Gestisce dipendenze e packaging in modo moderno
<code>uv</code>	Strumento moderno e molto veloce

Per ora, `venv` è più che sufficiente.

Verificare dove sei

```
# Su macOS/Linux: mostra quale Python sta usando
which python

# Su Windows: mostra quale Python sta usando
where python

# Mostra le librerie installate nell'ambiente attivo
pip list
```